

06. Explicación profunda del código

AI Dataset Foundry

Versión 0.2.0

2026-09-10

Fuente: 06-deep-code-explanation.md

06. Explicación profunda del código

Orquestación: `pipeline.build_dataset`

1. Recorre `config.inputs`. Cada locator se entrega a `ingest`; una excepción se convierte en `{input,error}` y no detiene otros locators.
2. Crea un único `Deduplicator`, por lo que la deduplicación cubre todos los documentos de la corrida.
3. Para cada documento ejecuta NFKC/espacios (`normalize_text`) y elimina NUL/controles/repeticiones (`clean_text`). Esto modifica el texto, no los originales.
4. `chunk_text` divide por ventana, oración, encabezado Markdown o párrafo. `_pack` agrupa unidades hasta el tamaño; una unidad sobredimensionada se corta sin overlap.
5. Cada pieza vacía se omite. La deduplicación ocurre antes de calidad: un fragmento rechazado queda registrado en el deduplicador y puede causar que una copia posterior, potencialmente con mejor provenance, sea eliminada. Es un riesgo de orden.
6. `assess_quality` calcula razones y score. Cualquier razón de rechazo hace `accepted=False`; el email sólo se agrega como flag y no rechaza.
7. Construye `ChunkRecord`: copia metadata, agrega `chunk_index`, hash del contenido transformado y referencia al hash de bytes de fuente.
8. Exporta formatos, arma manifest y lo escribe. Si SQLite está habilitado, escribe DB, agrega su ruta y reescribe el manifest.

Precondiciones: configuración Pydantic válida y permisos de lectura/escritura. Poscondición esperada: lista y manifest concordantes; no hay rollback coordinado si un exportador falla.

Adquisición y routing

`ingest` prioriza HTTP(S). Las URLs `.git` y GitHub sin sufijos PDF/HTML se tratan como repositorios; el resto como página. En filesystem, una carpeta con `.git` se trata completa como repo; otra carpeta recorre archivos recursivamente. `load_path` usa extensión, con imports diferidos para no exigir extras.

Git clona con `--depth 1 --filter=blob:none` a un temporal, selecciona extensiones y silencia errores por archivo. No fija commit, no procesa binarios y no aplica límite explícito. Web hace un fetch con user-agent; no es crawler, no implementa robots/cache/reintentos y debe tratarse como entrada no confiable.

Deduplicación

`simhash` tokeniza `\w+`, genera Blake2b de 64 bits por token y vota bit a bit. `hamming` cuenta bits distintos. `is_duplicate` prueba SHA-256 exacto y después compara linealmente el SimHash con todos los anteriores. Sólo añade el candidato si se acepta como no duplicado. Resultado: determinista para orden fijo; tiempo potencial $O(n^2)$, sensible a orden y a idiomas/tokenización.

UI/API y escritorio

`webapp.build` crea `run_id`, directorios exclusivos y guarda uploads en bloques de 1 MiB usando sólo el basename. El límite se comprueba durante escritura; al excederse, elimina la corrida. Decodifica locators/formats como arrays JSON y ejecuta el pipeline en thread. La descarga exige `run_id.isalnum()` y basename exacto para bloquear traversal.

`desktop.main` reserva un puerto loopback, inicia Uvicorn en thread daemon, espera hasta 5 segundos, abre pywebview y solicita salida al cerrar. Inferencia basada en el código: una terminación abrupta puede dejar el thread/proceso sin cierre elegante, aunque el daemon limita el bloqueo.

Persistencia/exportación

JSONL serializa un dict por línea; TXT pierde metadata; Parquet usa PyArrow/Zstd. SQLite crea dos tablas, índices, borra el contenido previo y realiza inserts en una transacción. `dataset_meta` conserva el manifest previo a que `pipeline` agregue la clave `sqlite`, porque la DB recibe el manifest antes de esa mutación; el archivo JSON sí incluye la clave. Esta discrepancia está registrada como deuda, no corregida.